

BiteBit: An Explicitly Typed Minimal
Language for Bitwise and Low-Level
Computation
[White Paper]

Executive Summary.....	2
Abstract.....	2
Introduction.....	3
Problem Statement.....	3
Core Language Features.....	3
Type System.....	3
Example of Syntax and Semantics.....	4
Interpreter / Compiler Architecture.....	4
Conclusion.....	4

Executive Summary

BiteBit is a small programming language made for bitwise operations, binary and hex values, and simple low-level numeric programming. The main goal of the language is to make this kind of work easier to read and easier to write without using a much larger language like C or MIPS.

Abstract

A lot of high-level languages support bitwise operations, but they do not make bit-level programming feel like the main purpose of the language. Lower-level languages are powerful, but they can also be harder to learn and use for small programs. BiteBit is meant to be a middle ground. It keeps the language small and focused while still supporting the features needed for bitwise and low-level computation.

Introduction

The idea behind BiteBit is to make bitwise programming more direct. Bitwise work shows up in systems programming, embedded programming, data formatting, and binary manipulation. Instead of adding bitwise features to a large general-purpose language, BiteBit puts them near the center of the language design.

Problem Statement

There is a gap between simple scripting languages and low-level languages. In Python or JavaScript, bitwise operators exist, but the language is not really built around them. In C or MIPS, low-level work is more natural, but there is more complexity and setup. BiteBit is meant to fill that gap by giving the user a smaller language that is easier to learn and still useful for bitwise tasks.

Core Language Features

BiteBit supports decimal, binary, and hexadecimal integer literals, bitwise operators such as `&`, `|`, `^`, `~`, `<<`, and `>>`, variables and assignment, control flow like `if`, `else`, and `while`, functions with `return`, and a small standard library with helpers like `print`, `bin`, and `hex`.

Type System

The original BiteBit draft described the language as dynamic, but the current preliminary direction moved toward a simpler, explicit type system using `Int`, `Bool`, `String`, and `Null`. The typed syntax was chosen to make declarations, validation, and symbol-table storage easier to handle.

Example of Syntax and Semantics

A simple BiteBit example is:

```
let x : Int = 0xABCD;
let low : Int = x & 0xFF;
let moved : Int = low << 8;
```

This example shows the main purpose of BiteBit because it uses hex values, masking, and shifting. The original BiteBit example used the same kind of low-byte isolation and shift operation to show the language's focus

Interpreter / Compiler Architecture

The planned BiteBit pipeline is still simple. The front end includes a lexer/tokenizer and a parser. After that, the language uses a symbol table and an execution stage to evaluate statements and expressions.

Conclusion

Overall, BiteBit is a small language designed for bitwise and low-level numeric work. The language is not meant to do everything. Instead, it is meant to stay simple, readable, and focused. That makes it a good fit for a compiler/interpreter course project and a good way to practice language design concepts like tokens, grammar, types, and symbol tables.